ONENET · WINDOWS SERVER · DATABASE ADMINISTRATION

HR & Payroll Database Commands

Every command your IT administrator needs to look after the HR & Payroll database on Windows Server: status, services, backups, restore, optimisation, connecting with pgAdmin or psql, ready-made queries, exports, logs and fixes.

Check	Protect	Tune	Look
Status, services, logs	Backup and restore	Optimise and monitor	pgAdmin, psql, queries

Database
PostgreSQL 16 (built in)

Port
5433, this server only

Issued
08 November 2026

Contents

1. Before you start
2. Where everything is
3. Status and services
4. Backups
5. Restoring a backup
6. Optimising the database
7. Connecting with pgAdmin 4
8. Connecting with psql
9. Ready-made queries
10. Exporting data
11. A separate copy for testing
12. Logs
13. Passwords and sign-in
14. Updating
15. Problems and fixes
16. Rules to keep the data safe

Before you start

1. Sign in to the server (HEADOFFICE) with an administrator account, directly or by Remote Desktop.
2. Open **Windows PowerShell as Administrator**: Start → type *PowerShell* → right-click → **Run as administrator**. Every command below is typed there.
3. This guide uses the HEADOFFICE folders `D:\HRPayroll` and `D:\HRPayrollData`. If HR & Payroll was installed somewhere else (the default is `C:\HRPayroll` and `C:\HRPayrollData`), change the drive letter in each command.

> **Copying commands**
> Type or paste one block at a time and press Enter. Lines starting with `#` are explanations and are ignored by PowerShell. In SQL, lines starting with `--` are explanations.

Where everything is

WHAT	WHERE / VALUE
Program folder	`D:\HRPayroll` (application in `app\<version>`)
Data folder	`D:\HRPayrollData` (database in `pgdata`, uploads in `documents`)
Tools (status, backup, restore, update, password reset)	`D:\HRPayroll\tools`
Database programs (psql, pg_dump, pg_restore)	`D:\HRPayroll\runtime\pgsql\bin`
Database server	host `localhost`, port **5433**, database **hr_payroll**. Accepts connections from this server only.
Database administrator	user `postgres`; password in `D:\HRPayroll\config\database-admin.txt`
Application database user	user `hr_app`; password inside `DATABASE_URL` in `D:\HRPayroll\config\.env` (between `hr_app:` and `@`)

Windows services	`HRPayrollDB` (database) and `HRPayrollApp` (application; depends on the database)
Nightly backup task	`HR Payroll Nightly Backup` (Task Scheduler)
Backups	`D:\HRPayrollData\backups`, files `hr-payroll-<date>.dump.enc` (encrypted, newest 14 kept)
Logs	`D:\HRPayrollData\logs`: `app`, `postgresql`, `backup.log`

> **The config folder is locked**
> Only Administrators can open `D:\HRPayroll\config`. To read a password file, open Notepad with **Run as administrator**, or in the administrator PowerShell: `notepad D:\HRPayroll\config\database-admin.txt`

Status and services

Overall status (services, health, addresses, database size, last backup, free disk):

```
powershell -ExecutionPolicy Bypass -File D:\HRPayroll\tools\hrpayroll-status.ps1
```

See the two services:

```
Get-Service HRPayrollDB, HRPayrollApp
```

Restart the application (most problems; the database keeps running):

```
Restart-Service HRPayrollApp
```

Restart everything, in the right order (stop the application first, start the database first):

```
Stop-Service HRPayrollApp
Stop-Service HRPayrollDB
Start-Service HRPayrollDB
Start-Service HRPayrollApp
```

Is the application answering? (change the port if yours is not 8080):

```
Invoke-WebRequest http://localhost:8080/api/health -UseBasicParsing | Select-Object StatusCode,
Content
```

Is the database listening?

```
& "D:\HRPayroll\runtime\pgsql\bin\pg_isready.exe" -h localhost -p 5433
```

Backups

A backup runs automatically every night. Run one by hand before any risky change:

```
powershell -ExecutionPolicy Bypass -File D:\HRPayroll\tools\hrpayroll-backup.ps1
```

List the backups, newest first:

```
Get-ChildItem D:\HRPayrollData\backups\hr-payroll-*.dump.enc | Sort-Object LastWriteTime -
Descending | Select-Object Name, LastWriteTime, @{n='MB';e={[math]::Round($_.Length/1MB,1)}}
```

Did last night's backup run?

```
Get-ScheduledTaskInfo -TaskName 'HR Payroll Nightly Backup' | Select-Object LastRunTime,
LastTaskResult, NextRunTime
Get-Content D:\HRPayrollData\logs\backup.log -Tail 20
```

`LastTaskResult` 0 means success. **Run the nightly task now:**

```
Start-ScheduledTask -TaskName 'HR Payroll Nightly Backup'
```

Copy backups to another disk or a network share (do this regularly; a backup on the same disk does not survive a disk failure):

```
robocopy D:\HRPayrollData\backups \\FILESERVER\Backups\HRPayroll hr-payroll-*.dump.enc /XO
```

> **Keep the backup key**
> Backups are encrypted with `BACKUP_ENCRYPTION_KEY` in `D:\HRPayroll\config\.env`. Keep a copy of that key somewhere safe outside the server: without it no backup can be restored.

Restoring a backup

A restore **replaces the whole database** with the backup: everything entered after that backup is lost. The tool takes a safety backup of the current database first, stops the application, restores, and starts it again.

```
powershell -ExecutionPolicy Bypass -File D:\HRPayroll\tools\hrpayroll-restore.ps1 -BackupFile
D:\HRPayrollData\backups\hr-payroll-2026-11-07T20-00-00-000Z.dump.enc
```

Type **RESTORE** when asked. Use the real file name from the backup list above.

Optimising the database

Normally nothing has to be done: every night after the backup a **quick** optimisation runs, and on the first Sunday of the month a **deep** one (shrink). The easiest way to run them by hand is in the application: **Data Backup** → **Database health & optimisation**.

Quick optimisation now (also takes a backup first; safe while people work):

```
powershell -ExecutionPolicy Bypass -File D:\HRPayroll\tools\hrpayroll-backup.ps1
```

Deep optimisation (shrink) now: run outside working hours, each table is locked while it is rewritten:

```
$env:HR_DEEP_OPTIMIZE = '1'
powershell -ExecutionPolicy Bypass -File D:\HRPayroll\tools\hrpayroll-backup.ps1
Remove-Item Env:HR_DEEP_OPTIMIZE
```

The result is written to `D:\HRPayrollData\logs\backup.log`. The same in SQL, for one table (in pgAdmin or psql, connected to **hr_payroll**):

```
-- quick: refresh statistics, make space reusable (no lock)
VACUUM (ANALYZE) attendances;
-- deep: rewrite and shrink (locks the table while it runs)
VACUUM (FULL, ANALYZE) attendances;
```

Connecting with pgAdmin 4

Run pgAdmin on the server itself (the database does not accept connections from other computers).

1. In pgAdmin: right-click **Servers** → **Register** → **Server...**
2. **General** tab: Name `HR Payroll (embedded)`.
3. **Connection** tab: Host `localhost`, Port **5433**, Maintenance database `hr_payroll`, Username `postgres` (or `hr_app`), Password from the files above. Tick *Save password* if you wish. **Save**.
4. Open the tree with the small arrows: **Servers** → **HR Payroll (embedded)** → **Databases** → **hr_payroll** → **Schemas** → **public** → **Tables**.
5. Right-click a table → **View/Edit Data** → **First 100 Rows**. To type a query: select **hr_payroll**, then **Tools** → **Query Tool**, paste the query and press **F5**.

SQL SERVER (SSMS)	PGADMIN 4
Server	Servers → HR Payroll (embedded)
Database	Databases → hr_payroll
Schema dbo	Schemas → public
New Query	Tools → Query Tool (F5 runs)
TOP 100	`LIMIT 100` at the end of the query

> **Where the data is**
> Each kind of record has its own table: `employees`, `attendances`, `users`, `roles`, `departments`, `leaves`, `payroll_runs`, `payroll_payslips`, `legacy_records` (imported CyberAP history), `hr_audit_log`, `app_settings`. The details of each record (name, code, CNIC, punch times ...) are in its **data** column, read like `data->>'fullName'`. The table `hr_documents` is from an older version and is not used.

Connecting with psql

psql is the command-line client included with the database. Start it (it asks for the password from `database-admin.txt`):

```
& "D:\HRPayroll\runtime\pgsql\bin\psql.exe" -h localhost -p 5433 -U postgres -d hr_payroll
```

Run one query without opening psql:

```
& "D:\HRPayroll\runtime\pgsql\bin\psql.exe" -h localhost -p 5433 -U postgres -d hr_payroll -c
"SELECT count(*) FROM employees;"
```

INSIDE PSQL	DOES
`\dt`	list tables
`\d employees`	columns of a table
`\l`	list databases
`\x`	switch to one-field-per-line display (good for the data column)
`\timing`	show how long each query takes
`\i D:\file.sql`	run the queries in a file
`\q`	quit

End every SQL statement with a semicolon `;`.

Ready-made queries

All of these only **read** data; they change nothing. They are also in `HR-Payroll-Database-Queries.sql` next to this PDF, ready to paste.

People

Record count of every table

```
SELECT relname AS table_name, n_live_tup AS records
FROM pg_stat_user_tables
ORDER BY n_live_tup DESC;
```

Employee list (first 100 by code)

```
SELECT id, data->>'employeeCode' AS code, data->>'fullName' AS name, data->>'cnic' AS cnic,
       COALESCE(data->>'status', 'Active') AS status, data->>'dateOfJoining' AS joined
FROM employees
ORDER BY data->>'employeeCode'
LIMIT 100;
```

Find an employee by name, code or CNIC (change the text)

```
SELECT id, data->>'employeeCode' AS code, data->>'fullName' AS name, data->>'cnic' AS cnic,
COALESCE(data->>'status', 'Active') AS status
FROM employees
WHERE data->>'fullName' ILIKE '%ahmed%'
   OR data->>'employeeCode' = '1001'
   OR data->>'cnic' = '35202-1234567-1';
```

Staff count by status

```
SELECT COALESCE(data->>'status', 'Active') AS status, count(*) AS staff
FROM employees
GROUP BY 1
ORDER BY 2 DESC;
```

Current staff by department

```
SELECT COALESCE(d.data->>'name', '(no department)') AS department, count(*) AS staff
FROM employees e
LEFT JOIN departments d ON d.id = e.data->>'departmentId'
WHERE COALESCE(e.data->>'status', 'Active') <> 'Terminated'
GROUP BY 1
ORDER BY 2 DESC;
```

Duplicate employee codes

```
SELECT data->>'employeeCode' AS code, count(*) AS times
FROM employees
WHERE COALESCE(data->>'employeeCode', '') <> ''
GROUP BY 1 HAVING count(*) > 1;
```

Duplicate CNICs

```
SELECT data->>'cnic' AS cnic, count(*) AS times
FROM employees
WHERE COALESCE(data->>'cnic', '') <> ''
GROUP BY 1 HAVING count(*) > 1;
```

Attendance

Attendance of one day (change the date)

```
SELECT e.data->>'employeeCode' AS code, e.data->>'fullName' AS name,
       COALESCE(a.status, a.data->>'status') AS status, a.data->>'punchIn' AS punch_in, a.data-
>>'punchOut' AS punch_out
FROM attendances a
JOIN employees e ON e.id = a.employee_id
WHERE a.date = '2026-11-08'
ORDER BY 1;
```

Totals by status for today

```
SELECT COALESCE(status, data->>'status') AS status, count(*) AS records
FROM attendances
WHERE date = current_date
GROUP BY 1
ORDER BY 2 DESC;
```

One employee, this month (change the code)

```
SELECT a.date, COALESCE(a.status, a.data->>'status') AS status, a.data->>'punchIn' AS punch_in,
a.data->>'punchOut' AS punch_out
FROM attendances a
JOIN employees e ON e.id = a.employee_id
WHERE e.data->>'employeeCode' = '1001'
  AND a.date >= date_trunc('month', current_date)
ORDER BY a.date;
```

Punched in yesterday but no punch out

```
SELECT e.data->>'employeeCode' AS code, e.data->>'fullName' AS name, a.data->>'punchIn' AS punch_in
FROM attendances a
JOIN employees e ON e.id = a.employee_id
WHERE a.date = current_date - 1
  AND COALESCE(a.data->>'punchIn', '') <> ''
  AND COALESCE(a.data->>'punchOut', '') = '';
```

Attendance records per month

```
SELECT to_char(date, 'YYYY-MM') AS month, count(*) AS records
FROM attendances
GROUP BY 1
ORDER BY 1 DESC;
```

Users, payroll, history

Login users and their roles

```
SELECT u.data->>'username' AS username, u.data->>'email' AS email,
       COALESCE(r.data->>'name', r.name, u.data->>'roleId') AS role, COALESCE(u.status, u.data-
>>'status') AS status
FROM users u
LEFT JOIN roles r ON r.id = COALESCE(u.role_id, u.data->>'roleId')
ORDER BY 2;
```

Roles

```
SELECT id, COALESCE(data->>'name', name) AS name, status
FROM roles
ORDER BY id;
```

Latest payslips

```
SELECT r.data->>'title' AS payroll_run, p.data->>'employeeCode' AS code, p.data->>'employeeName' AS
name,
       p.data->>'grossSalary' AS gross, p.data->>'totalDeductions' AS deductions, p.data-
>>'netSalary' AS net
FROM payroll_payslips p
JOIN payroll_runs r ON r.id = p.payroll_run_id
ORDER BY p.created_at DESC
LIMIT 50;
```

Imported CyberAP history, by kind

```
SELECT kind, count(*) AS records, min(record_date) AS first_date, max(record_date) AS last_date
FROM legacy_records
GROUP BY kind
ORDER BY kind;
```

Last 50 actions in the audit log

```
SELECT created_at, actor_uid, action, collection_name, document_id
FROM hr_audit_log
ORDER BY created_at DESC
LIMIT 50;
```

Saved settings

```
SELECT key, updated_at
FROM app_settings
ORDER BY key;
```

Health and performance

Database size

```
SELECT pg_size_pretty(pg_database_size(current_database())) AS database_size;
```

Largest tables and unused rows

```
SELECT relname AS table_name, pg_size_pretty(pg_total_relation_size(relid)) AS total_size,
       n_live_tup AS records, n_dead_tup AS unused_rows
FROM pg_stat_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 15;
```

Who is connected right now

```
SELECT pid, usename, client_addr, state, now() - query_start AS running_for, left(query, 80) AS
query
FROM pg_stat_activity
WHERE datname = current_database()
ORDER BY query_start;
```

Queries running for more than a minute

```
SELECT pid, now() - query_start AS running_for, left(query, 120) AS query
FROM pg_stat_activity
WHERE state = 'active' AND now() - query_start > interval '1 minute';
```

When tables were last cleaned and analysed

```
SELECT relname AS table_name, GREATEST(last_vacuum, last_autovacuum) AS last_cleaned,
       GREATEST(last_analyze, last_autoanalyze) AS last_analysed
FROM pg_stat_user_tables
ORDER BY n_live_tup DESC;
```

Memory settings

```
SELECT name, setting, unit
FROM pg_settings
WHERE name IN ('shared_buffers', 'effective_cache_size', 'work_mem', 'maintenance_work_mem',
'max_connections', 'port');
```

PostgreSQL version

```
SELECT version();
```

Exporting data

Easiest: the application's **Reports** menu exports every report to CSV (Excel) and PDF. For anything else, export a query to a CSV file with psql (one line):

```
& "D:\HRPayroll\runtime\pgsql\bin\psql.exe" -h localhost -p 5433 -U postgres -d hr_payroll -c
"\copy (SELECT data->>'employeeCode' AS code, data->>'fullName' AS name, data->>'cnic' AS cnic FROM
employees ORDER BY 1) TO 'D:\employees.csv' WITH (FORMAT csv, HEADER)"
```

In pgAdmin: run the query, then use the **Save results to file** button (download icon) above the results.

A separate copy for testing

To try something risky, make an **unencrypted** copy and load it into a separate database, so the live one is never touched. Delete the dump file afterwards: it contains salaries and CNICs in readable form.

```
$pg = "D:\HRPayroll\runtime\pgsql\bin"
# 1. dump the live database (asks for the postgres password)
& "$pg\pg_dump.exe" -h localhost -p 5433 -U postgres -Fc -f D:\hr_copy.dump hr_payroll
# 2. create an empty test database and load the copy into it
& "$pg\createdb.exe" -h localhost -p 5433 -U postgres hr_payroll_copy
& "$pg\pg_restore.exe" -h localhost -p 5433 -U postgres -d hr_payroll_copy --no-owner
D:\hr_copy.dump
# 3. when finished: remove the test database and the dump
& "$pg\dropdb.exe" -h localhost -p 5433 -U postgres hr_payroll_copy
Remove-Item D:\hr_copy.dump
```

> **Never**
> Never restore into or drop `hr_payroll` with these commands. For a real restore use `hrpayroll-restore.ps1` (above).

Logs

```
# application (newest file, last 50 lines)
Get-ChildItem D:\HRPayrollData\logs\app | Sort-Object LastWriteTime -Descending | Select-Object -
First 1 | Get-Content -Tail 50
# database
Get-ChildItem D:\HRPayrollData\logs\postgresql | Sort-Object LastWriteTime -Descending | Select-
Object -First 1 | Get-Content -Tail 50
# backups and optimisation
Get-Content D:\HRPayrollData\logs\backup.log -Tail 30
# follow a log live (Ctrl+C stops)
Get-Content D:\HRPayrollData\logs\backup.log -Tail 10 -Wait
```

Passwords and sign-in

Forgotten application administrator password:

```
powershell -ExecutionPolicy Bypass -File D:\HRPayroll\tools\Reset-AdminPassword.ps1 -Email
onenetpk@gmail.com -Generate
```

It prints a new password once; sign in with it and change it. Leave out `-Generate` to type your own.

Database passwords (`postgres` , `hr_app`): read them from the files in "Where everything is". Do not change them in pgAdmin: the application would no longer be able to connect.

Updating

Unzip the new package, then (change the version in the folder name):

```
cd D:\HRPayroll-Windows-Setup-2026.11.08-0.0.15\setup
powershell -ExecutionPolicy Bypass -File .\Update-HRPayroll.ps1 -InstallDir D:\HRPayroll
```

The update backs up the database first and rolls back by itself if the new version does not start. Afterwards press **Ctrl+F5** in each browser.

Problems and fixes

PROBLEM	TRY
The site does not open	`Get-Service HRPayrollDB, HRPayrollApp` ; if one is stopped, start the database first, then the application. Then read the application log.
Very slow	Run the "Queries running for more than a minute" and "Largest tables" queries. A high *unused_rows* count: run a deep optimisation at night.
A query is stuck	Find its `pid` with "Who is connected", then in SQL: `SELECT pg_cancel_backend(12345);` (stops the query) or `SELECT pg_terminate_backend(12345);` (closes that connection).
pgAdmin cannot connect	Use port **5433** (not 5432), host `localhost`, and run pgAdmin on the server itself. Check the database service is running.
"password authentication failed"	Copy the password again from the file (no spaces). `postgres` and `hr_app` have different passwords.
Disk filling up	Check free space with the status tool; old backups beyond 14 are removed automatically. Run a deep optimisation. Move old copies from `D:\HRPayrollData\backups` to another disk.
Backup failed	Read `D:\HRPayrollData\logs\backup.log`. Usually: disk full, or the database service stopped.

Rules to keep the data safe

DO	DO NOT
Use the application (screens, Reports) for daily work and changes.	Change records with `UPDATE` / `DELETE` in pgAdmin: it bypasses the application's checks and the audit log.
Take a backup before anything unusual.	Stop the database service while the application is running.
Keep copies of backups and of the backup key off the server.	Edit files in `pgdata` or the PostgreSQL settings by hand.
Sign in to pgAdmin as `hr_app` when you only want to look.	Share the `postgres` password or open port 5433 to the network.